

Little Pinger

A Windows desktop network diagnostics tool built with WPF / .NET 8. It bundles a wide range of network tools into a single tabbed interface — from simple ping monitoring to SSL certificate inspection and scheduled tests.

Requirements

OS	Windows 10 / 11
Runtime	.NET 8 (Windows)
Build	Visual Studio 2022 or <code>dotnet build</code>

Some features (ARP table, netstat, traceroute, Wi-Fi info) read data from Windows command-line utilities (`arp` , `netstat` , `tracert` , `netsh` , `ipconfig` , `route`). These are present on all modern Windows installations. Killing a process or scanning low-numbered ports may require administrator privileges.

Building

```
dotnet build LittlePinger.csproj
dotnet run --project LittlePinger.csproj
```

Or open `Little Pinger.slnx` in Visual Studio 2022 and press **F5**.

Tab overview

Tab	What it does
 Ping	Manage and run a list of continuous ICMP ping targets
 Monitoring & Analysis	Continuous ping graphs, interface bandwidth, ARP table, route table
 Wi-Fi & Local	Wi-Fi signal monitor, LAN device scanner, DHCP lease inspector

Tab	What it does
 Core Network Diagnostics	Traceroute, network info, DNS lookup, port scanner, netstat, speed test, MTU discovery
 Connectivity	HTTP/HTTPS check, SSL/TLS certificate inspector, WHOIS lookup
 Reporting	Export results, scheduled tests, historical comparison
 Settings	Configure which tabs and sub-tabs are visible

Features

Ping

The main tab. Maintains a persistent list of ping targets.

- **Add / Edit / Remove** entries (Ctrl+N / F2 / Del).
- Each entry shows: Name, IP / hostname, status (colour-coded), last RTT, sent / success / fail counts.
- **Start All / Stop All** to control the whole list at once, or toggle individual entries.
- Per-entry configurable **interval** and **timeout** (ms).
- Entries are saved to `Data/entries.json` and restored on next launch.
- Right-click any entry for quick actions.

Monitoring & Analysis

Continuous Ping

Multiple independent ping panels, each with its own host, interval and timeout settings.

- Live **latency graph** (up to 300 samples, rendered in code-behind).
- Stats per panel: sent / received, min / avg / max RTT, **jitter**, **packet loss %**.
- Add and remove panels independently. The host drop-down is pre-populated from the Ping tab.

Interface Monitor

Real-time bandwidth monitor for all non-loopback network adapters.

- Polls every second with a DispatcherTimer.
- Columns: **Name**, **Status**, **RX/s**, **TX/s**, **Total RX**, **Total TX**.
- Start / Stop live monitoring; single Refresh for a one-shot snapshot.

ARP Table

Reads the Windows ARP cache (`arp -a`).

- Columns: **IP Address**, **MAC Address**, **Type**, **Name** (resolved via reverse DNS in the background).
- Filters out multicast / broadcast entries automatically.
- Right-click any row → **Add to Ping List** (disabled if already present; uses the resolved hostname as the display name).

Route Table

Parses `route print` into separate IPv4 and IPv6 sections.

- **IPv4** columns: Network, Netmask, Gateway, Interface, Metric.
- **IPv6** columns: Network / Prefix, Gateway, Interface, Metric.
- Right-click any row → **Add to Ping List** (gateway IP used as both address and name).

Wi-Fi & Local

Wi-Fi Signal

Polls `netsh wlan show interfaces` on a configurable interval (default 2 s).

- Live fields: **SSID**, **BSSID**, **Radio type**, **Channel**, **Rx/Tx Mbps**, **Authentication**, **Signal %**.
- Signal strength bar:  Excellent →  Very Weak.
- Signal history graph (up to 300 samples). Gaps are shown for periods with no Wi-Fi connection.
- Start / Stop monitoring; single Refresh; Clear graph history.

LAN Scanner

Parallel ICMP ping-sweep of any subnet.

- Configurable **subnet** and **prefix length** (CIDR, up to /20 = 4 096 hosts).
- Auto-detects the local subnet on launch.
- Up to 50 concurrent probes for speed.
- After the sweep, enriches results from the ARP table (**MAC address**) and optionally resolves **hostnames** via reverse DNS.
- Columns: **IP**, **MAC**, **Hostname**, **Response (ms)**, **Status**.
- **Live search** filter across IP, MAC and hostname.
- Progress bar while scanning; Cancel button.
- Right-click → **Add to Ping List** (uses hostname as display name when available).

DHCP Leases

Parses `ipconfig /all` to show lease information for every active adapter.

- Columns: **Adapter, IP Address, Subnet Mask, Default Gateway, DHCP Server, Lease Obtained, Lease Expires, Days Left, DNS Servers, DHCP Enabled.**
- Right-click → **Add to Ping List.**

Core Network Diagnostics

Traceroute

Streams `tracert` output line-by-line into a text pane.

- Address drop-down pre-populated from Ping-tab entries and persisted history.
- Successfully traced addresses are saved to `Data/traceroute_history.json`.
- Cancel mid-trace.

Network Info

Reads all non-loopback network interfaces via the .NET `NetworkInterface` API.

- Columns: **Name, Description, Type, Status, MAC, IPv4, Subnet, Gateway, IPv6, DNS Servers, Speed.**
- Refresh button.

DNS Lookup

Wraps `nslookup` with support for **forward and reverse lookups** and selectable record types.

- **Record types:** AUTO, A, AAAA, PTR, MX, NS, TXT, CNAME, SOA.
AUTO resolves to PTR for IP inputs and A for hostnames.
- Optional **custom DNS server** override.
- Output streamed line-by-line. Cancel mid-lookup.
- History saved to `Data/dns_history.json` (max 50 entries).

Port Scanner

Scans TCP ports on a target host with configurable concurrency.

- **Port picker:** grouped, categorised port list (Web, Mail, Database, Remote, File Transfer, ...).
Tri-state group checkboxes; inline editing of port number and service name; custom ports.
- Configurable **concurrency** (simultaneous connections) and **timeout** (ms).
- Results grid: **Port, Service, Status** (Open / Closed / Filtered).
- Export results to CSV.
- Cancel mid-scan.

- Right-click open ports → **Add to Ping List**.

Netstat

Runs `netstat -ano` and maps PIDs to executable names.

- Columns: **Protocol, Local Address, Foreign Address, State, Process ID, Executable**.
- **Filter bar** (always visible):
- **Protocol** drop-down: All / TCP / UDP.
- **State** drop-down: populated dynamically from loaded data (ESTABLISHED, LISTENING, TIME_WAIT, etc.); resets if the current selection disappears after a refresh.
- **Search** text box: case-insensitive substring match across local address, foreign address, PID and executable name. Ports are part of the address string and are therefore searchable.
- ✕ **Clear** button resets all three filters at once.
- Status bar shows "X of Y connections (filtered)" when a filter is active.
- Right-click → **Kill Process** (with Yes/No confirmation dialog), **Add Local IP to Ping List**, **Add Foreign IP to Ping List**.

Speed Test

Measures latency, download, and upload speed against a selectable server.

- **Servers:** Cloudflare (50 MB download + 10 MB upload), Tele2 Europe (25 MB), OVH Europe (25 MB).
- **Phase 1 — Latency:** 5 ICMP pings → average and jitter.
- **Phase 2 — Download:** streamed HTTP GET with live Mbps progress.
- **Phase 3 — Upload:** Cloudflare only (10 MB POST).
- Live progress bar and status text. Cancel mid-test.

MTU Discovery

Finds the path MTU using ICMP probes with the DF (Don't Fragment) bit set.

- **Fast path:** tests the standard Ethernet payload (1 472 bytes → 1 500 MTU) first.
- **Binary search** from 0 – 1 472 bytes when the fast path fails.
- Live log of every probe with OK / Too large result.
- Discovered MTU displayed prominently below the log.
- History saved to `Data/mtu_history.json`.

Connectivity

HTTP / HTTPS Check

Performs an HTTP GET and surfaces the full response details.

- **Follow redirects** option (each hop shown in the redirect chain).
- Result card: **status code**, **response time (ms)**, **final URL**.
- Full **response headers** table.
- Colour-coded status group: Success (2xx), Redirect (3xx), Client Error (4xx), Server Error (5xx).
- History saved to `Data/http_history.json`.

SSL / TLS Inspector

Connects via `SslStream` (raw TLS, no HTTP) to read the server certificate and full chain.

- Configurable **port** (default 443).
- Certificate properties: Subject, Issuer, Serial, Thumbprint, Key Algorithm, Valid From/To, SANs, etc.
- **Certificate chain** table with per-certificate subject, issuer, expiry date and days remaining.
- Expiry banner with colour-coded severity: Good (>30 days), Warning (≤30 days), Critical (≤7 days), Expired.
- History saved to `Data/ssl_history.json`.

WHOIS Lookup

Two-step WHOIS: queries `whois.iana.org` for the authoritative server, then queries that server.

- Works for domain names and IP addresses (IANA refers to the correct RIR — ARIN, RIPE, APNIC, etc.).
- Raw output displayed in a scrollable text pane.
- History saved to `Data/whois_history.json` (max 50 entries).

Reporting

Export Results

Exports the current Ping-tab entries to a file and opens it immediately.

- **Formats:** HTML (styled report), JSON, CSV.
- Exported fields: Name, IP Address, Running, Sent, Success, Fail, Loss %, Last RTT, Status, Exported At.

Scheduled Tests

Runs diagnostic checks automatically at a set interval, even while you're working in other tabs.

- **Test types:** Ping, HTTP (status code check), Port (TCP connect check).
- Per-test configuration: name, host, type, port (for HTTP/Port), interval (minutes), enabled toggle, alert-on-failure flag.
- A `DispatcherTimer` fires every 30 seconds and runs any test whose interval has elapsed.

- Inline **Alerts** log shows failures with timestamps.
- Tests persist to `Data/scheduled_tests.json`.

Historical Comparison

Compare current ping statistics against a saved baseline.

- **Take Baseline:** snapshots all current ping entries (name, IP, stats, timestamp) to `Data/baseline.json`.
- **Compare:** diffs current stats against the baseline.
- Columns: Name, IP, Baseline Sent, Current Sent, Baseline Loss %, Current Loss %, Loss Δ, Baseline RTT, Current RTT.
- Loss delta is colour-coded: green (improved), red (worse), grey (neutral; threshold ± 2 %).

Cross-tab integration

Several tabs are linked to the Ping list for convenience:

- **Add to Ping List** context menu is available on every IP-bearing grid (ARP Table, Route Table, LAN Scanner, DHCP Leases, Netstat local/foreign). The menu item is **disabled** when the IP is already in the list.
When the source row has a resolved hostname, it is used as the display name; otherwise the IP address is used.
- **Address drop-downs** in Traceroute, DNS Lookup, MTU Discovery, HTTP Check, SSL Inspector, WHOIS, and Continuous Ping are pre-populated from the Ping tab's current entries.

Keyboard shortcuts (Ping tab)

Key	Action
Ctrl+N	Add new entry
F2	Edit selected entry
Delete	Remove selected entry

Data persistence

All data files live in the `Data/` folder next to the executable.

File	Contents
<code>entries.json</code>	Ping tab entries (name, IP, interval, timeout)
<code>settings.json</code>	Tab and sub-tab visibility preferences
<code>scheduled_tests.json</code>	Scheduled test definitions and last-run state
<code>baseline.json</code>	Historical comparison baseline snapshot
<code>traceroute_history.json</code>	Recently traced addresses
<code>dns_history.json</code>	Recent DNS lookup targets
<code>mtu_history.json</code>	Recent MTU discovery targets
<code>http_history.json</code>	Recent HTTP check URLs
<code>ssl_history.json</code>	Recent SSL inspector domains
<code>whois_history.json</code>	Recent WHOIS targets

⚙ Settings

Configure which tabs and sub-tabs appear in the main tab strip.

- The **Ping** tab is always visible and cannot be hidden.
- Click ▾ **Configure Visible Tabs** to open a dropdown tree view.
- Each top-level tab has a **bold checkbox**. Unchecking it hides that entire tab.
- Each top-level tab expands to show its **sub-tabs** as indented checkboxes. Unchecking a sub-tab hides it from the inner tab strip.
- Sub-tab checkboxes are **disabled** (greyed out) while their parent tab is hidden.
- Choices are saved immediately to `Data/settings.json` and restored on next launch.

Architecture

- **WPF / .NET 8** · `net8.0-windows` target framework, `UseWPF=true`.
- **Pure MVVM** — `ViewModelBase` implements `INotifyPropertyChanged`; `RelayCommand` provides `ICommand` with optional `CanExecute` predicate.
- All long-running work runs on `Task.Run` background threads; UI updates are marshalled back via `Application.Current.Dispatcher.Invoke`.

- `ICollectionView` (via `CollectionViewSource.GetDefaultView`) is used for client-side filtering in LAN Scanner and Netstat.
- Context-menu bindings that cross the `ContextMenu` visual-tree boundary use the `PlacementTarget.Tag` pattern: each `DataGrid` carries `Tag="{Binding DataContext, ElementName=RootWindow}"` to tunnel the root `MainViewModel` into menu item bindings.